
РАБОЧИЙ ПРОТОКОЛ И ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5
"ЛР 5: [C++ и UNIX]: C++ OOP / PARALLEL"

Группа: Z3344
Студенты: Поздеев Дмитрий
Преподаватель: Маслов И.

К работе допущен:
Работа выполнена:
Отчет принят:

1 Цель работы

- Познакомить студента с принципами объектно-ориентированного программирования на примере создания сложной синтаксической структуры. Придумать синтаксис своего персонального мини-языка параллельного программирования, а также реализовать его разбор и вычисление.

2 Задачи, решаемые при выполнении работы

- C++ PARALLEL LANG Создать параллельный язык программирования
- LOG Результат всех вышеперечисленных шагов сохранить в репозиторий (+ отчет по данной ЛР в папку doc)

3 Код

3.1 OOP

Листинг 1: Код на плюсах

```
#include <iostream>
#include <vector>
#include <memory>
#include <string>
#include <sstream>
#include <thread>
```

```

#include <mutex>
#include <map>
#include <chrono>
#include <fstream>
#include <cctype>
#include <stdexcept>

using namespace std::chrono;

//
class Expression {
public:
    virtual ~Expression() = default;
    virtual int evaluate(const std::map<std::string, int>& vars) const = 0;
};

//
class ConstantExpr : public Expression {
    int value;
public:
    ConstantExpr(int v) : value(v) {}
    int evaluate(const std::map<std::string, int>&) const override { return
        value; }
};

//
class VariableExpr : public Expression {
    std::string name;
public:
    VariableExpr(const std::string& n) : name(n) {}
    int evaluate(const std::map<std::string, int>& vars) const override {
        auto it = vars.find(name);
        return it != vars.end() ? it->second : 0;
    }
};

//
class BinaryOpExpr : public Expression {
    char op;
    std::unique_ptr<Expression> lhs, rhs;
public:
    BinaryOpExpr(char o, std::unique_ptr<Expression> l, std::unique_ptr<
        Expression> r)
        : op(o), lhs(std::move(l)), rhs(std::move(r)) {}
    int evaluate(const std::map<std::string, int>& vars) const override {
        int l = lhs->evaluate(vars);
        int r = rhs->evaluate(vars);
        switch(op) {
            case '+': return l + r;
            case '-': return l - r;
            case '*': return l * r;
            case '/': return r != 0 ? l / r : 0;
            default: throw std::runtime_error("Unknown operator");
        }
    }
};

```

```

    }
}
};

//
std::unique_ptr<Expression> parse_expression(std::vector<std::string>::
    iterator& it, const std::vector<std::string>::iterator& end);

std::unique_ptr<Expression> parse_primary(std::vector<std::string>::
    iterator& it, const std::vector<std::string>::iterator& end) {
    if (it == end) throw std::runtime_error("Unexpected␣end");
    std::string token = *it++;
    if (isdigit(token[0])) {
        return std::make_unique<ConstantExpr>(std::stoi(token));
    } else if (token == "(") {
        auto expr = parse_expression(it, end);
        if (it == end || *it++ != ")") throw std::runtime_error("Expected␣
            ')')");
        return expr;
    } else {
        return std::make_unique<VariableExpr>(token);
    }
}

std::unique_ptr<Expression> parse_term(std::vector<std::string>::iterator&
    it, const std::vector<std::string>::iterator& end) {
    auto expr = parse_primary(it, end);
    while (it != end && (*it == "*" || *it == "/")) {
        char op = (*it++)[0];
        auto rhs = parse_primary(it, end);
        expr = std::make_unique<BinaryOpExpr>(op, std::move(expr), std::
            move(rhs));
    }
    return expr;
}

std::unique_ptr<Expression> parse_expression(std::vector<std::string>::
    iterator& it, const std::vector<std::string>::iterator& end) {
    auto expr = parse_term(it, end);
    while (it != end && (*it == "+" || *it == "-")) {
        char op = (*it++)[0];
        auto rhs = parse_term(it, end);
        expr = std::make_unique<BinaryOpExpr>(op, std::move(expr), std::
            move(rhs));
    }
    return expr;
}

//
class Command {
public:
    virtual ~Command() = default;

```

```

        virtual void execute(std::map<std::string, int>& vars, std::mutex&
            io_mutex) const = 0;
};

//
class PrintCommand : public Command {
    std::unique_ptr<Expression> expr;
public:
    PrintCommand(std::unique_ptr<Expression> e) : expr(std::move(e)) {}
    void execute(std::map<std::string, int>& vars, std::mutex& io_mutex)
        const override {
        int value = expr->evaluate(vars);
        std::lock_guard<std::mutex> lock(io_mutex);
        std::cout << value << std::endl;
    }
};

//
class AppendCommand : public Command {
    std::string filename;
    std::unique_ptr<Expression> expr;
public:
    AppendCommand(const std::string& f, std::unique_ptr<Expression> e)
        : filename(f), expr(std::move(e)) {}
    void execute(std::map<std::string, int>& vars, std::mutex& io_mutex)
        const override {
        int value = expr->evaluate(vars);
        std::lock_guard<std::mutex> lock(io_mutex);
        std::ofstream file(filename, std::ios::app);
        if (file) file << value << std::endl;
    }
};

//
class LoopCommand : public Command {
    std::string var;
    std::unique_ptr<Expression> start, end;
    std::vector<std::unique_ptr<Command>> body;
public:
    LoopCommand(const std::string& v, std::unique_ptr<Expression> s, std::
        unique_ptr<Expression> e, std::vector<std::unique_ptr<Command>> b)
        : var(v), start(std::move(s)), end(std::move(e)), body(std::move(b)
            ) {}
    void execute(std::map<std::string, int>& vars, std::mutex& io_mutex)
        const override {
        int s = start->evaluate(vars);
        int e = end->evaluate(vars);
        for (int i = s; i <= e; ++i) {
            vars[var] = i;
            for (const auto& cmd : body) {
                cmd->execute(vars, io_mutex);
            }
        }
    }
};

```

```

    }
};

//
std::vector<std::unique_ptr<Command>> parse_commands(std::vector<std::string>::iterator& it, const std::vector<std::string>::iterator& end) {
    std::vector<std::unique_ptr<Command>> commands;
    while (it != end) {
        std::string cmd = *it++;
        if (cmd == "loop") {
            std::string var = *it++;
            if (*it++ != "from") throw std::runtime_error("Expected 'from'");
            auto start = parse_expression(it, end);
            if (*it++ != "to") throw std::runtime_error("Expected 'to'");
            auto end_expr = parse_expression(it, end);
            if (*it++ != "{") throw std::runtime_error("Expected '{'");
            auto body = parse_commands(it, end);
            if (it == end || *it++ != "}") throw std::runtime_error("Expected '}'");
            commands.push_back(std::make_unique<LoopCommand>(var, std::move(start), std::move(end_expr), std::move(body)));
        } else if (cmd == "print") {
            auto expr = parse_expression(it, end);
            commands.push_back(std::make_unique<PrintCommand>(std::move(expr)));
        } else if (cmd == "append") {
            std::string filename = *it++;
            filename = filename.substr(1, filename.size() - 2); //

            auto expr = parse_expression(it, end);
            commands.push_back(std::make_unique<AppendCommand>(filename, std::move(expr)));
        } else if (cmd == ";") {
            continue;
        } else {
            throw std::runtime_error("Unknown command: " + cmd);
        }
    }
}

return commands;
}

//
std::vector<std::string> tokenize(const std::string& line) {
    std::vector<std::string> tokens;
    std::string token;
    bool in_quote = false;
    for (char c : line) {
        if (c == '"') {
            in_quote = !in_quote;
            if (!in_quote) {
                tokens.push_back(token);
                token.clear();
            }
        }
    }
    if (!in_quote) {
        tokens.push_back(token);
    }
    return tokens;
}

```

```

        }
    } else if (in_quote) {
        token += c;
    } else if (isspace(c)) {
        if (!token.empty()) {
            tokens.push_back(token);
            token.clear();
        }
    } else {
        token += c;
    }
}
if (!token.empty()) tokens.push_back(token);
return tokens;
}

int main() {
    std::vector<std::vector<std::unique_ptr<Command>>> all_commands;
    std::mutex io_mutex;
    std::string line;

    std::cout << "Enter commands (type 'run' to execute):\n";
    while (std::getline(std::cin, line) && line != "run") {
        auto tokens = tokenize(line);
        auto it = tokens.begin();
        try {
            auto commands = parse_commands(it, tokens.end());
            all_commands.push_back(std::move(commands));
        } catch (const std::exception& e) {
            std::cerr << "Error: " << e.what() << std::endl;
        }
    }

    std::vector<std::thread> threads;
    std::vector<std::tuple<int, system_clock::time_point, system_clock::
        time_point>> timings;
    std::mutex timings_mutex;

    for (size_t i = 0; i < all_commands.size(); ++i) {
        threads.emplace_back([i, &all_commands, &io_mutex, &timings, &
            timings_mutex]() {
            auto start = system_clock::now();
            std::map<std::string, int> vars;
            for (const auto& cmd : all_commands[i]) {
                cmd->execute(vars, io_mutex);
            }
            auto end = system_clock::now();
            std::lock_guard<std::mutex> lock(timings_mutex);
            timings.emplace_back(i + 1, start, end);
        });
    }

    for (auto& t : threads) t.join();
}

```

```

std::cout << "\nExecution timings:\n";
for (const auto& [line_num, start, end] : timings) {
    std::cout << "Line_" << line_num
                << "_started_at_" << system_clock::to_time_t(start)
                << "_ended_at_" << system_clock::to_time_t(end)
                << "_("duration:_" << duration_cast<milliseconds>(end -
                    start).count() << "ms)\n";
}

return 0;
}

```

3.2 Работа с Git

Листинг 2: Клонирование репозитория и создание структуры

```

git clone https://github.com/PozdeevDA/my_labs.git
cd my_labs
mkdir -p lab_01/{build,src,doc,cmake} lab_02/{build,src,doc,cmake}
git checkout -b dev
git push -u origin dev
git branch -d main
git push origin --delete main

```

Листинг 3: Скрипт для переноса из dev в stg:

```

git add .
git commit -m "last_fix"
git push origin dev

```

4 Выводы

Лабораторная работа позволила освоить принципы ООП и параллельного программирования в C++. Реализован интерпретатор простого языка с поддержкой циклов, вывода и арифметики. Программа обрабатывает команды параллельно, используя потоки, и фиксирует время выполнения.